

Teoría de la Computabilidad

Módulo 13: Introducción a las funciones recursivas primitivas

Departamento de Cs. e Ing. de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

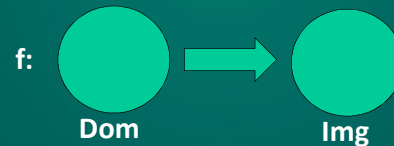
Funciones Recursivas Parciales (FRP)

- Teoría desarrollada por Kurt Gödel y Stephen Kleene.
- Las FRP brindan una propuesta práctica del concepto de computabilidad efectiva.
- Son una herramienta útil en la definición de funciones computables, pues son sencillas de manejar matemáticamente.
- Su equivalencia con funciones Turing Computables nos brinda una alternativa a la construcción de MTs.

¿ Por qué funciones ?

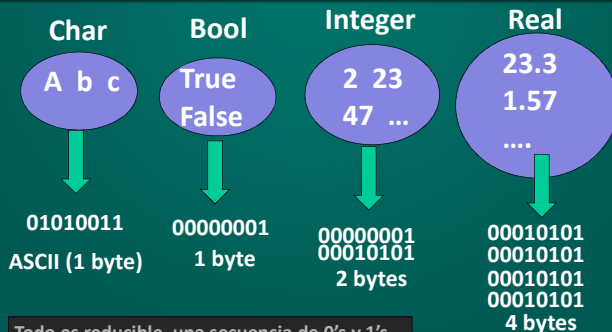
- La teoría de funciones tiene una larga historia en la matemática.
- El **Dominio** de una función puede asociarse los posibles **Datos de Entrada** de un algoritmo, y la **Imagen** a los posibles **Datos de Salida**.
- Pero... ¿cuál será la forma más sencilla de construir funciones? ¿Habrà alguna manera de construir todas las funciones discretas a partir de un conjunto de primitivas?

Desafío: ¿cómo representar funciones...?



Debemos pensar en representar todas las funciones posibles, con todos los dominios posibles y todas las imágenes posibles (¡ un gran desafío !)
Una manera de hacerlo: mapear los elementos del dominio y la imagen en algo fácilmente manejable y comprensible, como son **los números naturales**.

Reflexionando sobre los tipos de datos



Todo es reducible a una secuencia de 0's y 1's...
¡ que es mapeable a un número natural !

Algunos datos sobre Kurt Gödel



Nació el 28/4/1906 en Brunn, Imperio Austrohúngaro (hoy día Brno, en República Checa).
Murió el 14/01/1978 en Princeton, New Jersey, USA.

Conocido especialmente por sus "Teoremas de Incompletitud". En 1931 publicó estos resultados en el trabajo "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme."

Demostó resultados fundamentales, probando que en cualquier sistema matemático hay proposiciones que no pueden ser probadas o refutadas a partir de los axiomas del sistema.

Sus resultados fueron un hito en las matemáticas del siglo XX.
Murió en estado de demencia, creyendo que lo querían envenenar. Se negaba a comer, y murió por inanición.

Funciones Recursivas Primitivas

Funciones Recursivas primitivas: son funciones totales (definidas $\forall$ el dominio) de la forma
 $f: \text{Nat}^n \rightarrow \text{Nat}, n \geq 0$

definidas a partir de las llamadas funciones iniciales y dos reglas de construcción.

Funciones Iniciales:

- *cero, sucesor, identidad generalizada*

Reglas de construcción:

- composición, recursión primitiva

Definiremos fcs. recursivas primitivas tanto sobre números naturales como sobre cadenas de caracteres.

Función Recursiva Primitiva (FRPrim)

Def.: Una función es **recursiva primitiva** sobre *Nat* si es alguna de las siguientes **funciones base** o **iniciales** sobre *Nat*:

cero: $\text{Nat}^n \rightarrow \{0\}$, tal que $\text{cero}(x_1..x_n)=0$ para todo $x_1..x_n \in \text{Nat}^n, n \in \text{Nat}$

succ: $\text{Nat} \rightarrow \text{Nat}$, tal que $\text{succ}(x)=x+1, \forall n \in \text{Nat}$

Ident.Generalizada: $\Pi_i^n: \text{Nat}^n \rightarrow \text{Nat}$, definida por $\Pi_i^n(x_1..x_n) = x_i$

o bien puede generarse a partir de fcs. iniciales sobre *Nat* aplicando los **constructores**: **composición** o **recursión primitiva**.

IMPORTANTE: *Nat* representa el conjunto de los números naturales incluido el cero.

Funciones Base

cero: $\text{Nat}^n \rightarrow \{0\}$, tal que $\text{cero}(x_1..x_n)=0$ para todo $x_1..x_n \in \text{Nat}^n, n \in \text{Nat}$



succ: $\text{Nat} \rightarrow \text{Nat}$, tal que $\text{succ}(x)=x+1, \forall n \in \text{Nat}$



Ident.Gen.: $\Pi_i^n: \text{Nat}^n \rightarrow \text{Nat}$, $\Pi_i^n(x_1..x_n) = x_i$



Constructores: composición

Sean $m, n \geq 0$, la función total

$h: \text{Nat}^m \rightarrow \text{N}$,

y $g_i: \text{Nat}^n \rightarrow \text{N}$, con $0 \leq i \leq m$ una *familia* de funciones totales.

La función $f: \text{Nat}^n \rightarrow \text{N}$ puede definirse a partir de h y $g_1 \dots g_m$ como:

$$f(x_1..x_n) = h(g_1(x_1..x_n) \dots g_m(x_1..x_n)) = [h \circ (g_1, \dots, g_m)](x_1..x_n)$$

Decimos en tal caso que f resulta de las funciones g_i y h por **composición**.

Constructores: recursión primitiva

Sea $n \geq 0$,

g una fc. total recursiva primitiva n -aria,

y

h una fc. total recursiva primitiva $n+2$ -aria.

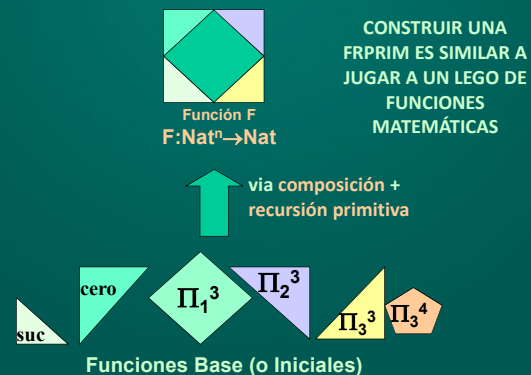
Puede definirse una fc. $n+1$ -aria f tal que, para toda tupla $(x_1..x_n) \in \text{Nat}^n$ y $m \in \text{Nat}$, se tiene:

$$f(x_1..x_n, 0) = g(x_1..x_n)$$

$$f(x_1..x_n, m+1) = h(x_1..x_n, m, f(x_1..x_n, m))$$

Decimos en tal caso que f resulta de g y h por **recursión primitiva**.

Cómo construir una FRPrim



Ejemplos de FRPrim

- $f: \text{Nat}^2 \rightarrow \text{Nat}$, $f(x,y) = y$ $\xrightarrow{\text{-----}} f(x,y) = \Pi_2^2(x,y)$
 - $\text{dos}: \text{Nat} \rightarrow \text{Nat}$, $\text{dos}(x) = 2$ $\xrightarrow{\text{-----}} \text{dos}(x) = \text{succ} \circ \text{succ} \circ \text{cero}(x)$
 - $\text{suma}: \text{Nat}^2 \rightarrow \text{Nat}$, $\text{suma}(x,y) = x+y$
- $\text{suma}(x,0) = \Pi_1^1(x)$
 $\text{suma}(x,y+1) = \text{succ} \circ \Pi_3^3(x,y, \text{suma}(x,y))$

13

Observación

Las funciones recursivas primitivas constituyen la menor clase de fcs. que contiene a las iniciales y es cerrada bajo composición y recursión primitiva.

Ninguna otra función (excepto las fcs. base y las construibles según las reglas anteriores) son funciones recursivas primitivas sobre Nat .

14

Derivación formal de una función f

Def.: Una **derivación formal de la función f** será toda secuencia de funciones $g, h, \dots, f$ tal que cada miembro de la secuencia es:

- una función base;
- es obtenida por aplicar alguna de las fcs. constructoras a miembros anteriores de la secuencia.

15

Ejemplo 1

Definiremos la función constante $\text{cuatro}: \text{Nat} \rightarrow \{4\}$ donde $\text{cuatro}(a)=4$, $\forall a \in \text{Nat}$, en términos de **cero**, **composición**, y **sucesor**.

Componiendo cuatro veces con sucesor, a partir de cero, obtenemos:

$$\text{cuatro}(x) = \text{succ} \circ \text{succ} \circ \text{succ} \circ \text{succ} \circ \text{cero}(x)$$

La derivación formal recursiva primitiva de **cuatro** para este ejemplo es:

- | | |
|---|----------------|
| (1) succ , | función base |
| (2) $\text{cero}(x)$, | función base |
| (3) $\text{succ} \circ \text{cero}(x)$, | comp.(1) y (3) |
| (4) $\text{succ} \circ \text{succ} \circ \text{cero}(x)$, | comp.(1) y (3) |
| (5) $\text{succ} \circ \text{succ} \circ \text{succ} \circ \text{cero}(x)$, | comp.(1) y (4) |
| (6) $\text{succ} \circ \text{succ} \circ \text{succ} \circ \text{succ} \circ \text{cero}(x) \equiv \text{cuatro}$ | comp.(1) y (5) |

Ej 3: Puede probarse $f: \text{Nat}^3 \rightarrow \text{Nat}$, donde $f(a,b,c)=c+1$, $\forall (a,b,c) \in \text{Nat}^3$ es recursiva primitiva. En efecto, puede comprobarse que podemos escribirla en términos de sucesor, id. generalizada, y composición:

$$h = \text{succ} \quad m = 1 \quad n = 3 \quad g_1 = \Pi_3^3$$

$$f(x_1, x_2, x_3) = h(g_1(x_1, x_2, x_3)) = \text{succ}(\Pi_3^3(x_1, x_2, x_3))$$

En este caso, se obtuvo la derivación formal recursiva primitiva: " Π_3^3 , succ , f " de la fc. f .

- | | |
|--|----------------|
| (1) succ , | función base |
| (2) $\Pi_3^3(x_1, x_2, x_3)$, | función base |
| (3) $\text{succ} \circ \Pi_3^3(x_1, x_2, x_3)$ | comp.(1) y (3) |

17

Ej 4: Puede probarse que $\text{suma}: \text{Nat}^2 \rightarrow \text{Nat}$, donde $\text{suma}(a,b)=a+b$, $\forall a,b \in \text{Nat}$ es una fc. recursiva primitiva. Basta definir $\text{suma}(a,b)$ usando id. generalizada, recursión, sucesor y composición:

$$\text{suma}(x_1, 0) = \Pi_1^1(x_1)$$

$$\text{suma}(x_1, x_2 + 1) = \text{succ}(\Pi_3^3(x_1, x_2, \text{suma}(x_1, x_2))) =$$

$$= (\text{succ} \circ \Pi_3^3)(x_1, x_2, \text{suma}(x_1, x_2))$$

Nota: usualmente escribiremos solo x_1 en lugar de $\Pi_1^1(x_1)$ para simplificar la notación usada.

18

$\text{suma}(x_1, 0) = \Pi_1^1(x_1)$
 $\text{suma}(x_1, x_2 + 1) = \text{suc}(\Pi_3^3(x_1, x_2, \text{suma}(x_1, x_2))) =$
 $= (\text{suc} \circ \Pi_3^3)(x_1, x_2, \text{suma}(x_1, x_2))$

Ej: $\text{suma}(2, 2) = \text{suma}(2, 1+1) =$
 $\text{suc}(\Pi_3^3(2, 1, \text{suma}(2, 1))) =$
 $\text{suc}(\Pi_3^3(2, 1, \text{suma}(2, 0+1))) =$
 $\text{suc}(\Pi_3^3(2, 1, \text{suc}(\Pi_3^3(2, 0, \text{suma}(2, 0))))) =$
 $\text{suc}(\Pi_3^3(2, 1, \text{suc}(\Pi_3^3(2, 0, \Pi_1^1(2))))) =$
 $\text{suc}(\Pi_3^3(2, 1, \text{suc}(\Pi_3^3(2, 0, 2)))) =$
 $\text{suc}(\Pi_3^3(2, 1, \text{suc}(2))) = \text{suc}(\Pi_3^3(2, 1, 3)) =$
 $\text{suc}(\Pi_3^3(2, 1, 3)) = \text{suc}(3) = 4$

19

Ej 5: Puede probarse que la fc. **predecesor** $\text{pred: Nat} \rightarrow \text{Nat}$, es una fc. recursiva primitiva. Predecesor se define como:

$$\text{pred}(a) = \begin{cases} 0 & \text{si } a=0 \\ a-1 & \text{si } a \geq 1 \end{cases}$$

Luego es expresable a través de la notación de fc. recursivas primitivas de la sgte. forma:

$$\begin{aligned} \text{pred}(0) &= \text{cero}() \\ \text{pred}(y+1) &= \Pi_1^2(y, \text{pred}(y)) \end{aligned}$$

20

Ej 6: En base al ej.5, podemos probar que la **resta** o **diferencia primitiva** $\text{difp: Nat}^2 \rightarrow \text{Nat}$, es una fc. recursiva primitiva. **difp** se define como la resta habitual cuando $a-b$ es un nro. natural, y 0 cuando $a-b < 0$. Esto es:

$$\text{difp}(a, b) = \begin{cases} 0 & \text{si } a < b \\ a-b & \text{si } a \geq b \end{cases}$$

Análogamente al caso de la suma, podemos pensar que $5-3 = 5-(2+1)$, o equivalentemente $5-2-1$. En gral, resulta $a-(b+1) = (a-b)-1$. Resulta:

$$\begin{aligned} \text{difp}(x, 0) &= \Pi_1^1(x) \\ \text{difp}(x, y+1) &= (\text{pred} \circ \Pi_3^3)(x, y, \text{difp}(x, y)) \end{aligned}$$

21

$$\text{difp}(a, b) = \begin{cases} 0 & \text{si } a < b \\ a-b & \text{si } a \geq b \end{cases}$$

$$\begin{aligned} \text{difp}(x, 0) &= \Pi_1^1(x) \\ \text{difp}(x, y+1) &= \text{pred}(\Pi_3^3(x, y, \text{difp}(x, y))) \end{aligned}$$

Ej: $\text{difp}(3, 2) = \text{difp}(3, 1+1) =$
 $\text{pred}(\Pi_3^3(3, 1, \text{difp}(3, 1))) =$
 $\text{pred}(\Pi_3^3(3, 1, \text{difp}(3, 0+1))) =$
 $\text{pred}(\Pi_3^3(3, 1, \text{pred}(\Pi_3^3(3, 0, \text{difp}(3, 0))))) =$
 $\text{pred}(\Pi_3^3(3, 1, \text{pred}(\Pi_3^3(3, 0, \Pi_1^1(3))))) =$
 $\text{pred}(\Pi_3^3(3, 1, \text{pred}(\Pi_3^3(3, 0, 3)))) =$
 $\text{pred}(\Pi_3^3(3, 1, \text{pred}(3))) =$
 $\text{pred}(\Pi_3^3(3, 1, 2)) = \text{pred}(2) = 1$

22

Ej 7: Podemos probar que la fc. **difabs: Nat² → Nat**, notada $|a-b|$, es una fc. recursiva primitiva.

$$|a - b| =_{\text{def}} \begin{cases} (a-b) & \text{si } a \geq b \\ (b-a) & \text{si } a < b \end{cases}$$

Podemos lograr definirla en términos de fcs. recursivas primitivas del sgte. modo:

$$\text{difabs}(x, y) = \text{suma}(\text{difp}(x, y), \text{difp}(y, x))$$

Luego $|a-b|$ es fc.rec.prim., pues resulta expresable usando dos fc.rec.primativas (suma y difp) combinadas por composición.

23

Ej 8: Sea $f: \text{Nat} \rightarrow \text{Nat}$, definida como $f(a) = \text{if } (a > 0) \text{ then } a \text{ else } 0$. Veamos que f es fc.rec.prim:

$$\begin{aligned} f(0) &= \text{cero}() \\ f(y+1) &= (\text{suc} \circ \Pi_1^2)(y, f(y)) \end{aligned}$$

Ej 9: Sea **sumatoria: Nat → Nat**, definida como $\text{sumatoria}(a) = \sum_{i=0..a} i$. También es fc.rec.prim. En efecto, $\text{sumatoria}(a+1) = (a+1) + \text{sumatoria}(a) = 1 + (a + \text{sumatoria}(a))$. Luego:

$$\begin{aligned} \text{sumatoria}(0) &= \text{cero}() \\ \text{sumatoria}(y+1) &= \text{suc}(\text{suma}(y, \text{sumatoria}(y))) \\ &= (\text{suc} \circ \text{suma})(y, \text{sumatoria}(y)) \end{aligned}$$

24

Algunos ejemplos simples

Func. Rec. Primitivas

- $f(a)=a+2, \forall a \in \text{Nat}$
- $f(a,b,c)=c+1, \forall (a,b,c) \in \text{Nat}^3$
- $\text{suma}(a,b)=a+b, \forall a,b \in \text{Nat}$
- $\text{pred}:\text{Nat} \rightarrow \text{Nat},$
 $\text{pred}(a)=a-1$ si $a>0$; $\text{pred}(a)=0$ si $a=0$
- $\text{difp}:\text{Nat}^2 \rightarrow \text{Nat},$
 $\text{difp}(a,b)=a-b$ si $a \geq b$, $\text{difp}(a,b)=0$ si $a < b$.
- $\text{difabs}:\text{Nat}^2 \rightarrow \text{Nat},$
 $\text{difabs}(a,b)=a-b$ si $a \geq b$,
 $\text{difabs}(a,b)=b-a$ si $a < b$.

25

Algunos ejemplos simples

Func. Rec. Primitivas

- $f:\text{Nat} \rightarrow \text{Nat},$
 $f(a)=\text{if } (a>0) \text{ then } a \text{ else } 0, \forall a \in \text{Nat}$
- $\text{sumatoria}:\text{Nat} \rightarrow \text{Nat},$
 $\text{sumatoria}(a)=\sum_{i=0..a} i, \forall a \in \text{Nat}$
- Fc. constante $K_j^n:\text{Nat}^n \rightarrow \text{Nat},$
 $K_j^n(x_1, x_2, \dots, x_n)=j, \forall n \in \text{Nat}^n.$
- $\text{sg}:\text{Nat} \rightarrow \{0,1\},$
 $\text{sg}(a)=0$ si $a=0$ y $\text{sg}(a)=1$ si $a>0, \forall a \in \text{Nat}$

26

Función constante n-aria

Def.: Denominaremos **constante n-aria** a la fc. $K_j^n:\text{Nat}^n \rightarrow \text{Nat}$, para todo $n \geq 0$ y $j \geq 0$, tal que $K_j^n(x_1, x_2, \dots, x_n)=j$ para cualquier n-upla $n \in \text{Nat}^n$.

Ejemplo:

$K_{100}^n(x_1, x_2)=100$, para todo $(x_1, x_2) \in \text{Nat}^2$

Lema: Las fc. constantes son recursivas primitivas.

(demostrar como ejercicio. Pistas: usar las fcs. cero, suc, composición e inducción)

27

Ejemplo

En base a la fc. anterior, podemos mostrar que la fc. **signo** es recursiva primitiva, donde $\text{sg}:\text{Nat} \rightarrow \{0,1\}$ se define como sigue

$$\text{sg}(a) =_{\text{def}} \begin{cases} 0 & \text{si } a = 0 \\ 1 & \text{si } a > 0 \end{cases}$$

En efecto, podemos definirla mediante:

$\text{sg}(0) = \text{cero}()$
 $\text{sg}(y+1) = K_1^2(y, \text{sg}(y))$

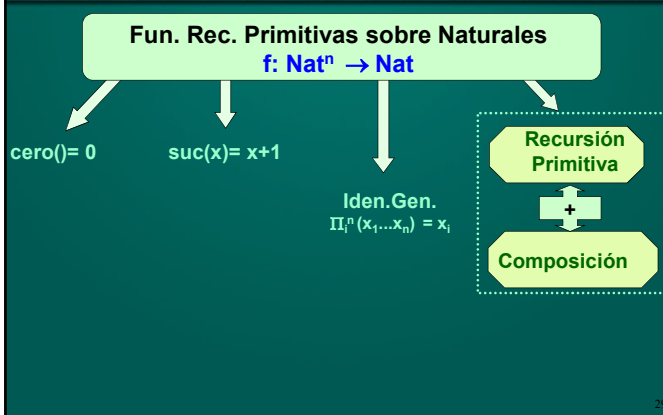
CUIDADO:

Recordar que no puede haber más de un caso base



28

Síntesis



29

Predicados Recursivos Primitivos

Def.: Denominaremos **predicado n-ario** $P(x_1, \dots, x_n)$ a una relación sobre n números naturales, es decir $P(x_1, \dots, x_n) \subseteq \text{Nat}^n$

Def.: La **fc. característica** de un predicado n-ario es la fc. n-aria $f:\text{Nat}^n \rightarrow \{0,1\}$ tal que para toda n-upla $(x_1, \dots, x_n) \in \text{Nat}^n$ se tiene:

$$f_P(x_1, \dots, x_n) =_{\text{def}} \begin{cases} 1 & \text{si } P(x_1, \dots, x_n) \text{ es verdadero} \\ 0 & \text{si } P(x_1, \dots, x_n) \text{ es falso} \end{cases}$$

Un predicado P se dice **predicado recursivo primitivo** (p.r.p.) sssi su fc. característica lo es.

30

Pred. Recursivos Primitivos: Ejemplo

Considérese $\text{menor} \subseteq \text{Nat}^2$, tal que

$$\text{menor}(x,y) = \{x,y \in \text{Nat} \mid x < y\}.$$

Entonces $\text{menor}(2,3) = \text{v}$, $\text{menor}(1,1) = \text{f}$.

La función característica $f_{\text{menor}}: \text{Nat}^2 \rightarrow \{0,1\}$ se comportará como sigue:

$$\begin{aligned} f_{\text{menor}}(2,3) &= 1 \\ f_{\text{menor}}(1,1) &= 0 \end{aligned}$$

Importante: para saber si un predicado es un p.r.p. debemos analizar **siempre** su función característica (si ésta es rec.primitive, entonces el predicado es p.r.p.)



31

Pred. Recursivos Primitivos: Ejemplo

Puede demostrarse que $\text{igual}(a,b)$ es un p.r.p. Para esto debemos mostrar que su fc. característica f_{igual} lo es. Informalmente podemos verlo como sigue.

Sea $f_{\text{igual}}: \text{Nat} \times \text{Nat} \rightarrow \{0,1\}$ y sea

$$f_{\text{igual}}(x,y) = 1 \div \text{sg}(\text{difabs}(x,y)) \text{ o más formalm.:}$$

$$f_{\text{igual}}(x,y) = \text{difp} \circ (K_1^2, \text{sg} \circ \text{difabs}(\Pi_1^2, \Pi_2^2))(x,y)$$

$$\text{Ej: } f_{\text{igual}}(2,2) = 1 \div \text{sg}(\text{difabs}(2,2)) = 1 \div 0 = 1 \text{ pero}$$

$$f_{\text{igual}}(2,7) = 1 \div \text{sg}(\text{difabs}(2,7)) = 1 \div 1 = 0$$

Como la fc. característica f_{igual} devuelve 1 cuando $y=x$ y 0 cuando $y \neq x$, ent. igual es un predicado recursivo primitivo.

32

Lema para Predicados Rec. Prim

Lema: Si P y Q son predicados recursivos primitivos, también lo son $\neg P$, $P \vee Q$, $P \wedge Q$.

(demostrar como ejercicio. Pistas: usar las operaciones aritméticas de resta, suma y producto)

Ejemplo 1:

Los predicados $\text{menor}(x,y) = \{x,y \in \text{Nat} \mid x < y\}$ e $\text{igual}(x,y) = \{x,y \in \text{Nat} \mid x = y\}$ son predicados recursivos primitivos. Luego $\text{menor}(x,y) \vee \text{igual}(x,y) = \{x,y \in \text{Nat} \mid x \leq y\}$ es un predicado recursivo primitivo.

Ejemplo 2:

El predicado $\text{men_o_ig}(x,y) = \{x,y \in \text{Nat} \mid x \leq y\}$ es un p.r.p. Luego $\text{mayor}(x,y) = \neg \text{men_o_ig}(x,y) = \{x,y \in \text{Nat} \mid x > y\}$ es un predicado recursivo primitivo.

33

Predicados recursivos primitivos disjuntos entre sí

Def.: Sean $P_1 \dots P_m$ pred. rec. primitivos. Diremos que son **disjuntos entre sí** cuando para toda n -upla $(x_1, x_2, \dots, x_n)$ se verifica que $(x_1, x_2, \dots, x_n)$ no pertenece a más de una relación determinada por los predicados $P_1 \dots P_m$.

Ejemplo:

Los predicados recursivos primitivos $\text{mayor}(x,y)$ y $\text{menor_o_igual}(x,y)$ son disjuntos entre sí.

34

Función definida por casos

Def.: Sean $P_1 \dots P_m$ predicados disjuntos entre sí, y sean $g_1, g_2, \dots, g_m$ fcs. recursivas primitivas. Denominaremos **función definida por casos** a toda fc. f con la sgte. estructura:

$$f(x_1 \dots x_n) = \begin{cases} g_1(x_1, x_2, \dots, x_n) & \text{si } P_1(x_1 \dots x_n) \\ \dots & \dots \\ g_m(x_1, x_2, \dots, x_n) & \text{si } P_m(x_1 \dots x_n) \end{cases}$$

Ejemplo:

$$f(x_1, x_2) = \begin{cases} \text{suma}(x_1, x_2) & \text{si } \text{mayor}(x_1, x_2) \\ \text{difabs}(x_1, x_2) & \text{si } \text{men_o_ig}(x_1, x_2) \end{cases}$$

35

Lema para funciones definidas por casos

Lema: Si $f(x_1, x_2, \dots, x_n)$ es una fc. definida por casos, entonces es **función recursiva primitiva**.

Demostración:

Queda como ejercicio. :)

Pista: definir f usando las fcs. características de los predicados considerados, de tal forma que anulen los valores asociados a cada fc. g_i cuando el predicado es falso para la n -upla dada. Tener en cuenta que los predicados son mutuamente excluyentes.

36

Lema - Ejemplo de aplicación

Puede mostrarse que la función $\max: \text{Nat}^2 \rightarrow \text{Nat}$, que obtiene el máximo elto. de un par, es función recursiva primitiva definiéndola por casos

$$\max(x,y) = \begin{cases} \Pi_1^2(x,y) & \text{si } x > y \\ \Pi_2^2(x,y) & \text{si } x \leq y \end{cases}$$

Los predicados ' $<$ ' y ' $\leq$ ' son primitivos recursivos, ident. generalizada también (por def.) y una fc. definida por casos es rec. primitiva.

Luego podemos concluir que $\max(x,y)$ es fc. recursiva primitiva.

37

Lema - Ejemplo de aplicación

Se puede llegar al mismo resultado que el ejemplo anterior usando composición de fc. recursivas primitivas. Una posible fc. característica de $x > y$ es $\text{sg}(x \div y)$, mientras que si consideramos la fc. $\text{sg}': \text{Nat} \rightarrow \{0,1\}$

$$\text{sg}'(x) = \begin{cases} 1 & \text{si } x=0 \\ 0 & \text{si } x>0 \end{cases}$$

Recordar que:
 $\text{sg}: \text{Nat} \rightarrow \{0,1\}$, $\text{sg}(a)=0$ si $a=0$ y $\text{sg}(a)=1$ si $a>0$

resulta $\text{sg}'(x)$ recursiva primitiva, dado que
 $\text{sg}'(x) = \neg \text{sg}(x)$

Obs.: si $\text{sg}'(x) = 1$ ent. $\text{sg}(x)=0$, y viceversa.

38

Lema - Ejemplo de aplicación

En consecuencia $\text{sg}'(x \div y)$ es una fc. característica de $x \leq y$.

$$\text{Luego } \max(x,y) = x * \underbrace{\text{sg}(x \div y)}_{1 \text{ ssi } x > y} + y * \underbrace{\text{sg}'(x \div y)}_{1 \text{ ssi } y \geq x}$$

sg y sg' son mutuamente excluyentes

Más formalmente:

$$\max(x,y) = \text{suma}^\circ(\text{producto}^\circ(\Pi_1^2, \text{sg} \circ \text{difp}(\Pi_1^2, \Pi_2^2)), \text{producto}^\circ(\Pi_2^2, \text{sg}' \circ \text{difp}(\Pi_1^2, \Pi_2^2)))(x,y)$$

Nota: obsérvese que el ejemplo anterior puede realizarse con $f_{\leq}(x,y) = f_{\neg}(f_{>}(x,y))$

39